

ETHOnline 2026 Finalists

What the eight best projects built, why they stood out, and what we can use

How to use this guide

Part 1 gives the eight projects at a glance, ranked by how relevant they are to us.

Part 2 explains each project the same way: problem, example, how it works, what stood out, and **what we can borrow**. Read the three High ones closely; skim the rest.

Part 3 lists what all eight did well, then the **feature menu** and the **changes we suggest** for our project.

Part 4 has questions to discuss as a team, and a glossary.

Prepared 2 Oct 2026 from the ETHGlobal showcase submissions, the ETHOnline 2026 finale stream and our earlier deep-dive. WHU deadline: 5 Oct 2026, 00:00 CEST.

Part 1 · The eight finalists at a glance

ETHOnline 2026 ran online from 4 to 16 September with more than 800 submissions. Eight teams were chosen as finalists and demoed live. Relevance below means **how much the project can teach our own project**, not how good it is.

Project	In one line	Category	Sponsor prize	Relevance
Cordon	One shared budget for a whole team of AI agents	Agent spending control	—	High
OpenBook	Fresh blockchain data, or your money back	Data marketplace	The Graph 3rd; Arc Agentic Economy	High
OnchainRouter	OpenRouter, but for blockchain tools	Paid tool marketplace	—	High
Novi Corpus	A legal company, guardian and treasury for each AI agent	Agent identity + treasury	World AgentKit Continuity	Medium
Petri	Evolve agent setups through verified experiments	AI evaluation	—	Low
LeekWallet	The wallet decides what it is willing to sign	Hardware wallet / security	—	Low
TARE	Measure exactly what a Uniswap v4 hook costs you	DeFi measurement	Uniswap Best Stack Contribution	Low
ETH Arcade	The ETH price chart becomes a handheld arcade game	Consumer hardware game	—	Low

The one thing to remember

Cordon is our own idea, done on another chain, and it reached the finals. Our Hedera version (Capability Descent) had the same mechanism and did not place. The difference was not the code: Cordon stated the problem plainly, gave a dollar example, designed safety in visibly, and proved it with attack tests. Part 2 shows exactly how.

How they fit together

Seen together, the eight form pieces of an economy where AI agents act on their own. Ours sits in **money controls**, next to Cordon, and touches **data** and **tools** because our agents buy them.

- **Data** they can trust: OpenBook
- **Tools** they can pay for: OnchainRouter
- **Money controls**: Cordon
- **Identity and accountability**: Novi Corpus
- **Getting better over time**: Petri
- **Safe signing**: LeekWallet
- **Measuring protocols**: TARE
- **A consumer experience**: ETH Arcade

Part 2 · Project by project

Each project follows the same layout. The green box at the end of each is the part to act on.

1. Cordon

“One shared budget for a whole team of AI agents” Relevance to us: **High**

Why it matters to us

This is our own mechanism (Capability Descent) presented better. Same tree of agents, same rule that authority only shrinks going down. Cordon reached the finals; we did not place. Study how they framed and proved it.

THE PROBLEM

- AI agents choose tools, retry failed calls and hand work to other agents, so cost is hard to predict.
- If every agent gets its own separate allowance, the allowances together can exceed what the owner meant to spend.
- Fixed per-agent allowances also waste money: one worker sits on unused budget while another runs out.

EXAMPLE

The owner gives a research task **\$10**. The main agent hands work to two helpers. Helper A spends \$2, helper B spends \$3, the main agent needs \$4. All of it comes out of the same \$10. A purchase that would go over what is left is refused.

HOW IT WORKS

1. The owner sets a budget (a “mandate”) for the task.
2. Each agent can give a helper a smaller mandate, never a bigger one.
3. When any agent buys something, the cost is subtracted from that agent **and every agent above it, up to the owner** (“ancestor debit”). The tightest budget in the chain decides.
4. Over budget means the payment is refused and the refusal is recorded.

WHAT STOOD OUT

- **The agent has no “pay” or “transfer” tool at all.** It can only fetch a URL; the daemon pays.
- **Price and payee are read from the seller’s 402 response, never from the agent,** so a tricked agent cannot redirect money.
- Contracts have no admin key and cannot be upgraded.
- **Attack testing with numbers:** 1,200 adversarial strategies over 45,360 draws, none beat the budget. A hostile AI agent given \$0.02 managed to spend \$0.007 before being stopped.
- The problem is backed by a source: a June 2026 paper listing 63 real budget-overrun incidents.

BUILT WITH

Solidity contracts on Arc testnet (MandateRegistry, TreeVault, ConductRecord); TypeScript daemon with viem; x402 payments in USDC via Circle Gateway; Privy wallets; MCP tool `cordon_fetch(url)`.

What we can borrow

- Keep our core/ tree, but **pitch it Cordon’s way**: one plain problem, one dollar example, then the mechanism.
- **Remove any generic pay tool from our agent interface.** Read price and recipient from the 402 challenge.
- **Run a hostile-agent test** and put the number on a slide (“told to drain the budget, stopped at X”).
- Use “ancestor debit” as the name for what our tree already does; it is easier to explain.

2. OpenBook

"The data marketplace for AI agents. Fresh data or your money back." Relevance to us: **High**

Why it matters to us

Finalist and two sponsor prizes. It shows that "agents buying data" can win when it has a sharp promise (freshness) and automatic recourse (refund).

THE PROBLEM

- An agent may spend real money on data that turns out to be out of date, and act on it.
- Normal data APIs give no automatic refund when the data is stale.
- Machine buyers need discovery, payment, delivery and checking without a human.

EXAMPLE

An agent asks for a protocol's current lending rates. Payment is locked in escrow. The seller delivers data stamped with the block it came from. If that block is recent enough, the seller is paid; if not, the buyer gets the money back.

HOW IT WORKS

1. Buyer pays into escrow and the freshness rule is fixed up front.
2. Seller delivers the data; a hash of it is recorded.
3. The delivery carries a signed note of which block the data came from.
4. The contract compares that block with the freshness rule: fresh pays the seller, stale refunds the buyer.

WHAT STOOD OUT

- A **promise anyone understands**: fresh or your money back.
- The rule is enforced **at payment time by the contract**, not just claimed in marketing.
- It adds a trust layer on top of The Graph instead of competing with it.
- Opened the pitch with market size and real-world stats before any tech.

BUILT WITH

Solidity/Foundry on Arc testnet (custom SlaHook); The Graph subgraphs for data and block numbers; TypeScript/Bun MCP server and CLI; React front end; Circle wallets and x402; ENS names for sellers; 409 unit tests + 20 contract tests.

What we can borrow

- Give our product **one promise in plain words** (e.g. "your agent can never spend more than you allow").
- Copy the **pitch order**: size of the problem → one painful example → our promise → demo.
- Freshness check / refund is a good **roadmap** item; too big for our deadline.

3. OnchainRouter

“OpenRouter, but for blockchain tools” Relevance to us: **High**

Why it matters to us

Finalist. A clean example of pay-per-use for agents with a strong rule: no answer, no charge.

THE PROBLEM

- Developers have to wire up many separate blockchain APIs.
- An agent needs a way to pay tiny amounts without a human logging into a dashboard.
- Agents need answers (“is this safe?”), not raw tables.

EXAMPLE

A DeFi agent wants to know if lending conditions are good. It finds a lending-rate tool, sees the price, pays for one call and gets a structured answer. Then it calls a liquidation-risk tool without signing up anywhere.

HOW IT WORKS

1. Agent connects once (MCP or HTTP) and sees every tool and its price for free.
2. It calls a tool; the tool answers with HTTP 402 and a price.
3. Agent pays per call with x402; money goes straight to the tool’s author.
4. If the tool fails to answer, the payment is never settled.

WHAT STOOD OUT

- **No answer, no charge:** unsuccessful calls are never settled.
- No commission and no wallet held by the router.
- One registry generates the catalogue, docs and /llms.txt, so they never drift apart.
- Coverage is tested, not just claimed.

BUILT WITH

Next.js + TypeScript; remote MCP server; Zod schemas for each tool; x402 on Hedera (HBAR) and Arc (USDC); The Graph standard subgraphs; Claude Code integration.

What we can borrow

- **No answer, no charge** is cheap to copy in a simple form: only settle after a valid response.
- Show **price before purchase** to the agent and the human in the demo.
- An MCP tool list that installs into Claude Code made the demo easy to try; we already have MCP tools.

4. Novi Corpus

"A Wyoming legal company for each AI agent" Relevance to us: **Medium**

Why it matters to us

Finalist with a business framing: it ties agents to the real world (law, a named human). Their treasury also has spending limits, so it overlaps with us.

THE PROBLEM

- Agents that buy things and sign deals need someone accountable behind them.
- Counterparties need to check who they are dealing with.
- Owners need limits so the agent cannot spend freely.

EXAMPLE

A travel AI that books flights and hotels gets its own legal company, a verified human guardian, a treasury with spending rules and an identity other agents can check before doing business with it.

HOW IT WORKS

1. A human guardian verifies with World ID; only then can a company be formed.
2. The agent gets an on-chain identity (ERC-8004 / ENS) and a treasury.
3. The guardian can pause the treasury or replace the agent's key.
4. Other agents pay a small fee to check "is this a real company in good standing?" and get a signed answer.

WHAT STOOD OUT

- Combines **legal, human and financial accountability** in one product.
- A clear real-world buyer: businesses running agents that make purchases.

BUILT WITH

Solidity factory + treasury on Arc; World ID and AgentKit; Hedera x402 for the paid check; TypeScript/Hono + SQLite; MCP server; Next.js.

What we can borrow

- **Frame our user as a real business**, not a crypto user.
- A **guardian "pause everything" button** is cheap and demos well.
- Signed, checkable answers are a nice touch for receipts.

5. Petri

“Evolve AI agent setups through verified experiments” Relevance to us: **Low**

Why it matters to us

Different problem, but a model for how to present evidence.

THE PROBLEM

- Teams tweak prompts and agent code by feel, without proof the change helped.

EXAMPLE

Starting setup solved 5 of 20 coding tasks. A version with better context solved 19 of 20 and was accepted by two independent checkers.

HOW IT WORKS

1. Make a small change to the agent setup.
2. Run old and new five times each; compare the median.
3. Two different keys must confirm a big enough gain; the author cannot confirm their own change.
4. Keep failed attempts too, so nobody repeats them.

WHAT STOOD OUT

- Treats improvement as a **controlled experiment** with clear accept/reject rules.
- Everything can be replayed without an API key.

BUILT WITH

TypeScript/Node; Zod; Ed25519 signatures; hash-chained log; Claude via the Anthropic SDK; Next.js; Hedera and World.

What we can borrow

- **Write down our acceptance rules** (what counts as “refused correctly”) and show results as numbers.

6. LeekWallet

“The hardware wallet decides what it is willing to sign” Relevance to us: **Low**

Why it matters to us

A \$15 hardware wallet. Not our area, but its core rule matches ours.

THE PROBLEM

- Computers and phones can be hacked, so the final “sign?” decision should not happen there.

EXAMPLE

A dapp asks the wallet to sign. The device decodes the transaction itself, shows it on its own screen, and refuses anything it cannot understand unless you switch on blind signing.

HOW IT WORKS

1. Decode the transaction on the device.
2. Show it on the device screen.
3. Sign only after a physical button press; refuse unknown types by default.

WHAT STOOD OUT

- **Refuse by default:** unknown means no.

- Very cheap open hardware with heavy testing.

BUILT WITH

ESP32-S3 chip, small OLED screen, four buttons; firmware in C; desktop companion in Rust/Tauri; WalletConnect.

What we can borrow

- **Refuse by default** for our agents: an unknown seller or unknown request type is refused unless a human allows it.

7. TARE

“Measure exactly what a Uniswap v4 hook costs you” Relevance to us: **Low**

Why it matters to us

DeFi research tool. Useful only as an example of rigorous evidence.

THE PROBLEM

- A Uniswap v4 pool can run extra code (a hook) on every swap, and nobody states what it actually costs the trader.

EXAMPLE

Run the same swap twice on an identical copy of the chain, once with the real hook and once with the hook swapped for code that does nothing. The difference is what the hook takes.

HOW IT WORKS

1. Copy Base at a fixed block.
2. Quote a swap with the real hook.
3. Replace only the hook with an empty one.
4. Quote again and compare.

WHAT STOOD OUT

- A **controlled before/after test**.
- Marks “not measurable” honestly instead of writing zero.

BUILT WITH

Python engine on an Anvil fork; Uniswap quoter; 125,072 measurements across 7,817 pools; React front end; browser extension; Hedera x402.

What we can borrow

- Report what we **could not prove** as clearly as what we did (our RESULTS.md habit already does this).

8. ETH Arcade

“The ETH price chart becomes a handheld arcade game” Relevance to us: **Low**

Why it matters to us

Consumer hardware game. Shows that a physical, fun demo stands out.

THE PROBLEM

- Trading feels abstract; live market data rarely becomes a social, physical experience.

EXAMPLE

Turn a knob to put a box over a price range, stake USDC, and if ETH ends inside the box after ten seconds you win.

HOW IT WORKS

1. Live ETH price from The Graph.
2. Player stakes USDC (held in a contract on Arc).
3. Ten-second rounds run on the device; only funding and payouts go on chain.

WHAT STOOD OUT

- A **memorable live demo**.
- Fast play off chain, money on chain: a practical split.

BUILT WITH

Raspberry Pi Zero, 3.5" screen, rotary knob, 3D-printed case; Python firmware and SDK; Solidity escrow on Arc; ENS for player names and scores.

What we can borrow

- Keep **fast things off chain and money on chain**. Make the demo something people remember.

Part 3 · What to take into our project

What all eight did well

Pattern	Seen in	What it means for us
They lead with the problem, not the technology	Cordon, OpenBook	Our deck opens with one sentence a non-engineer understands, then one example in numbers.
They give one concrete example with real numbers	Cordon (\$10 task), Petri (5/20 → 19/20)	Write our own one-line example before building the demo.
They prove claims with tests and numbers	Cordon (1,200 attacks), OpenBook (409 tests), TARE (125k rows)	Run a hostile-agent test; put one number on a slide.
Safety is designed in, not bolted on	Cordon (no pay tool), LeekWallet (refuse by default)	Agent never chooses price or recipient; unknown means no.
A simple promise users remember	OpenBook (fresh or refund), OnchainRouter (no answer, no charge)	Pick our promise and repeat it everywhere.
They combine existing standards instead of inventing a protocol	All eight	Use what Solana already has (SPL delegation, Squads limits, Memo, Blinks).
Everything shown is actually deployed	All eight	Real devnet transactions with explorer links in the demo.

Feature menu

These work for **any** of our candidate use cases. Priority assumes the 5 Oct deadline: **Must** = core demo, **Should** = adds a strong moment, **Could** = if time allows, **Roadmap** = slide only. Effort: Small = hours, Medium = about a day, Large = more than we have.

Feature	Borrowed from	Effort	How on Solana	Priority
Shared budget with sub-limits (ancestor debit)	Cordon	Already in core/	SPL approve per budget account, or Squads spending limits	Must
Agent never chooses price or recipient	Cordon	Small	Read amount and payTo from the x402 challenge; agent only names the URL	Must
Refusal before payment, with reason codes	Cordon, ours	Already in core/	Chain also refuses (delegate cap)	Must
Approved sellers + max price per purchase	Cordon, LedgerMind	Small	Check before signing; new codes VENDOR_NOT_APPROVED, PRICE_ABOVE_MAX	Must
Receipt on every payment (who, what, why)	OpenBook, LedgerMind	Small	Memo instruction in the same transaction; report built from chain	Must
Human approval above a threshold	Leash, Harness, Novi Corpus	Medium	Solana Blink link: one tap to approve	Should

Feature	Borrowed from	Effort	How on Solana	Priority
Pause everything / revoke one agent	Novi Corpus, ours	Small	SPL revoke; others keep working	Should
Hostile-agent test with a number	Cordon	Medium	Script an agent told to overspend; log where it was stopped	Should
Refuse by default (unknown seller = no)	LeekWallet	Small	Allowlist check is the default path	Should
Don't pay twice for the same thing	Keryx, Carpool	Small	Lookup in our server before paying	Could
No answer, no charge	OnchainRouter	Medium	Settle only after a valid response	Could
Freshness rule with refund	OpenBook	Large	Escrow program; needs Anchor	Roadmap
Seller deposit lost on non-delivery	Our research notes	Large	Escrow + slashing program	Roadmap

The core demo, in any use case

1. An owner sets a budget and approved sellers. **2.** The agent buys three things inside the budget; each payment carries a receipt memo. **3.** An unapproved seller is refused before any money moves. **4.** A big purchase waits; the owner approves with one tap on a Blink. **5.** A spending report appears, built from Solana transactions. **6.** One agent is revoked; the others keep working.

Suggested changes to our current plan

Our plan today is in Analysis/strategy.md and Analysis/use-case-and-value-prop-2026-10-02.md. Based on the finalists, we suggest:

Today	Change to
Pitch opens with the mechanism (caps, revocation, x402)	Open with one plain problem and a dollar example, Cordon-style. Mechanism comes third.
The agent gets a pay function	Agent only requests a URL. Our server reads price and recipient from the 402 and pays, or refuses.
"Our tests pass"	One adversarial run with a number on a slide: what a hostile agent tried and where it was stopped.
Refusal codes OVER_LIMIT, REVOKED, PARENT_REVOKED	Add VENDOR_NOT_APPROVED, PRICE_ABOVE_MAX, NEEDS_APPROVAL. Unknown seller is refused by default.
Payment is just a transfer	Every payment carries a memo (budget, purpose). The spending report is read from the chain.
Approval is a dashboard idea	Approval is a Solana Blink: one tap from a phone. This is a feature only Solana has.
Many features attempted	Five "Must" features done well; everything else on the roadmap slide.
ENS / ERC-8004 identity from the Hedera version	Drop it. WHU judges do not score sponsor tech; Solana's role is what matters.

Today	Change to
Use case still open	Decide first; the feature list above works for any use case. Then write the one-line example.

Where we can be different

- **Solana enforces the limit.** Cordon used Arc; nothing at ETHOnline ran on Solana. On Solana the delegate cap is checked by the token program itself, so even a hacked server cannot overspend.
- **One-tap approval with Blinks.** No finalist had a phone approval link built into the chain's own tooling.
- **A spending report built from the chain.** Memos on every payment make an invoice or report that anyone can check line by line. No winner produced per-client or per-project reports.
- **A business user.** Following Novi Corpus, we pitch to a company with a real workflow, not to crypto users.

These are our reading of the submissions; check them against the projects before claiming them on a slide.

Part 4 • Questions for the team

Work through these before building further. Each answer feeds the deck or the demo.

1. What is our one promise, in under ten words, that a WHU business student understands?
2. Which use case do we pick, and who is the first real customer we could name?
3. What is our “\$10 task” example: one budget, two agents, one refusal, in real numbers?
4. Cordon is our mechanism on another chain and it reached the finals. What will a judge see that is different about ours? (Solana enforcement, Blink approval, chain-built report?)
5. Which of the five Must features can we show live on devnet, and which only in slides?
6. What number will our hostile-agent test produce, and is it convincing?
7. If a judge asks “why not just a company card?”, what is our two-sentence answer?
8. What do we put on the roadmap slide so the project looks like it can grow?

Glossary

Term	Plain meaning
AI agent	AI software that takes actions (calls tools, pays) instead of only answering.
Budget tree	A chain of budgets: owner → agent → helper agents. Each level gets at most what is above it.
Ancestor debit	Cordon’s rule: a helper’s purchase is subtracted from every budget above it.
x402	A way to pay for a web request: the server answers “402 Payment Required” with a price, the client pays and retries.
MCP	Model Context Protocol: a standard way for AI apps like Claude to find and call tools.
Escrow	Money held by a contract until agreed conditions are met.
Refuse by default	Anything not explicitly allowed is rejected.
SPL approve / revoke	Solana’s built-in way to let another key spend up to an amount from a token account, and to cancel that.
Squads spending limits	Solana multisig feature: amount, period, allowed recipients.
Memo	A Solana instruction that attaches a short text note to a transaction.
Blink	A Solana link that opens a ready-to-sign transaction, e.g. one-tap approval.
Devnet	Solana’s free test network.

Sources

- ETHGlobal showcase, ETHOnline 2026: ethglobal.com/showcase?events=ethonline2026
- ETHOnline 2026 Finale stream (16 Sep 2026): youtube.com/watch?v=ZGlvHtWMXZs
- Project pages: ethglobal.com/showcase/ followed by [cordon-vw3kh](#), [openbook-8ngw6](#), [onchainrouter-8r4jm](#), [novi-corpus-qtfxd](#), [petri-mjy1y](#), [leekdiyhardwarewallet-5qssg](#), [tare-ozced](#), [eth-arcade-96wyn](#)
- Our repo: [Analysis/ethonline-2026-winners-and-competitors-2026-10-02.md](#) (all winners and the 812-project screen)